Construtores em Python

Inicialização de objetos e boas práticas

```python
class Line:
    def __init__(self, p1, p2):
        self.line = (p1,p2)

    def what(self):
        print('Object is a line')

class Dot:
    def __init__(self,p1):
        self.point = p1

    def what(self):
        print('Object is a dot')

```

O Método __init__

O que é o método __init__?

Em Python, o `__init__` é um método especial (dunder method) que funciona como **construtor** de classe.

- É chamado **automaticamente** quando um objeto é criado
- Inicializa os **atributos de instância** do objeto
- Garante que o objeto esteja em um **estado válido** após sua criação

Quando é executado?

O método `__init__` é executado **após** a criação do objeto (após o método `__new__`), mas antes que o objeto seja retornado ao chamador.

O Método __init__

```python
class Pessoa:
  def __init__(self, nome, idade):
    self.nome = nome
    self.idade = idade

  def falar(self):
    print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")

```

```python
maria  = Pessoa("Maria",30)
maria.falar()
```

```
Olá, meu nome é Maria e tenho 30 anos.
```

Estrutura Básica

Definição de um Construtor

A estrutura básica de um construtor em Python consiste em definir o método **__init__** dentro da classe, com o parâmetro **self** como primeiro argumento, seguido pelos parâmetros que a classe precisa para inicialização.

Exemplo de Uso

```
#criando uma instância
obj = Minha_Classe("valor1",42)
#acessando os atributos
print(obj.atributo1) #valor1
print(obj.atributo2  #42
```

Dica: Os atributos definidos no construtor estão disponíveis em todos os métodos da classe através do parâmetro **self**

Sintaxe do Construtor

```
class Minha_Classe:
  def __init__(self,parametro1,parametro2):
    #atributos de instância
    self.atributo1 = parametro1
    self.atributo2 = parametro2
```

O Parâmetro self

O que é o parâmetro self?

O self é uma referência à **instância atual** da classe. É o primeiro parâmetro de qualquer método de instância em Python, incluindo o construtor **__init__** .

Características importantes

É apenas uma convenção de nomenclatura (poderia ter outro nome)

É passado automaticamente quando o método é chamado

Permite acessar atributos e métodos da instância

Por que é necessário?

O self permite que Python:

Diferencie entre atributos de instância e variáveis locais

Acesse o mesmo objeto em diferentes métodos

Mantenha o estado do objeto entre chamadas de métodos

Exemplo de uso do self

```python
class Pessoa:
  def __init__(self, nome, idade):
    self.nome = nome
    self.idade = idade

  def falar(self):
    print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")

  def aniversario(self):
    self.idade+=1
```

Chamada: p = Pessoa("Ana", 30)

↓

Python executa: Pessoa.__init__(p, "Ana", 30)

Onde 'p' é passado automaticamente como 'self'

Construtores sem Parâmetros

O que são?

Construtores sem parâmetros são métodos **__init__** que não recebem argumentos adicionais além do parâmetro **self** que é obrigatório.

São úteis quando a classe não precisa de dados externos para inicialização ou quando todos os atributos podem ser inicializados com valores padrão.

```
1 class SemParametros:
2     def __init__(self):
3         self.mensagem = "Olá"
4         self.contador = 0
5         self.ativo = True
```

Mesmo sem parâmetros adicionais, o construtor ainda pode inicializar atributos com valores predefinidos ou calculados.

Exemplo de Uso

```
7 #Criando uma instância
8 obj = SemParametros()
9 print(obj.mensagem) # Olá
10 print(obj.contador) # 0
11 print(obj.ativo)    # True
```

Caso de Uso: Contador

```
1 class Contador:
2     def __init__(self):
3         self.valor=0
4     def incrementar(self):
5         self.valor+=1
6     def obter_valor(self):
7         return self.valor
```

Neste exemplo, o contador sempre começa em 0 e não precisa de parâmetros para inicialização.

Valores Padrão para Parâmetros

O que são valores padrão?

Valores padrão permitem que parâmetros do construtor sejam opcionais, tornando a classe mais flexível e fácil de usar.

```
1 class Pessoa:
2   def __init__(self,nome,idade=0,email=None):
3     self.nome = nomes
4     self.idade = idade
5     self.email = email
```

Exemplos de uso

```
7   #Apenas o nome (obrigatório)
8   p1 = Pessoa("Ana")
9
10  #nome e idade
11  p2 = Pessoa("João",23)
12
13  #todos os parâmetros
14  p3 = Pessoa("Maria",18,"maria@gmail.com")
```

Benefícios dos valores padrão

Flexibilidade: Permite criar objetos com diferentes configurações sem precisar de múltiplos construtores.

Compatibilidade: Facilita a evolução da classe, adicionando parâmetros sem quebrar código existente.

Legibilidade: Torna o código mais claro ao definir valores sensatos para parâmetros opcionais

Cuidado com objetos mutáveis como valores padrão

```
18  #Problema NÃO FAÇA ISSO
19  def __init__(self,itens=[])
20      self.itens = itens
21
22  #solução correta
23  def __init__(self,itens=None):
24      self.itens = itens id itens is not None else []
```

Métodos de Fábrica (Factory Methods)

O que são Factory Methods?

Python não suporta sobrecarga de construtores como outras linguagens. Para simular **múltiplos construtores**, usamos **métodos de classe** decorados com **@classmethod**.

- **Construtores alternativos** que permitem criar objetos a partir de diferentes fontes ou formatos de dados
- Recebem **cls** como primeiro parâmetro (referência à classe) em vez de **self**
- Retornam uma nova instância da classe usando **cls()** ou **return cls(...)**

Quando usar Factory Methods?

Quando precisa criar objetos a partir de diferentes formatos de dados

Para implementar padrões de projeto como Singleton ou Builder

Para melhorar a legibilidade do código com nomes descritivos

Exemplo de Factory Method

```
class Pessoa:
def __init__(self, nome, idade):
 self.nome = nome
 self.idade = idade

@classmethod
def from_string(cls, dados):
 nome, idade_str = dados.split(",")
return cls(nome, int(idade_str))

@classmethod
def from_dict(cls, dicionario):
return cls(dicionario['nome'], dicionario['idade'])
```

Uso dos Factory Methods

```
# Usando o construtor padrão
p1 = Pessoa("Ana", 30)
```

__new__ vs __init__

Diferenças Fundamentais

Característica	__new__	__init__
Função	Cria a instância	Inicializa a instância
Ordem de execução	Primeiro	Segundo
Parâmetro inicial	cls (classe)	self (instância)
Retorno	Deve retornar a instância	Não deve retornar valor (None)
Uso comum	Casos avançados (Singleton, imutáveis)	Uso cotidiano

Na prática: Quase sempre usamos apenas __init__ para inicializar objetos em Python.

```python
class Singleton:
    __instance = None

    def __new__(cls):
        if cls.__instance is None:
            cls.__instance = object.__new__(cls)
        return cls.__instance

s1 = Singleton()
s2 = Singleton()

print(s1)
print(s2)
```

✓ 0.0s

```
creating...
<__main__.Singleton object at 0x000001C12D23C280>
<__main__.Singleton object at 0x000001C12D23C280>
```

Fluxo de execução: Quando criamos um objeto com `obj = MinhaClasse()`, Python primeiro chama __new__ para criar

Melhores Práticas

Mantenha o __init__ simples

Foque apenas na inicialização de atributos. Evite lógica complexa, operações de I/O ou chamadas de rede.

Use parâmetros significativos

Dê nomes claros e descritivos aos parâmetros para melhorar a legibilidade do código.

Defina valores padrão

Sempre que possível, forneça valores padrão para parâmetros opcionais.

Evite efeitos colaterais

A inicialização de um objeto não deve afetar o estado de outros objetos de forma inesperada.

Cuidado com objetos mutáveis

Má prática

```
class MinhaClasseRuim:
def __init__ (self, lista=[]):
 self.lista = lista
 self.lista.append( "item")

 obj1 = MinhaClasseRuim()
 obj2 = MinhaClasseRuim()
# obj1.lista e obj2.lista apontam para o mesmo objeto!
```

Boa prática

```
class MinhaClasseBoa:
def __init__ (self, lista=None):
 self.lista = lista if lista is not None else []
 self.lista.append( "item")
```

Construtores em Python: Conclusão

Pontos-chave

- O método__**init**__ é o construtor padrão em Python para inicializar objetos
- O parâmetro**self** é uma referência à instância atual do objeto
- Valores padrão tornam os construtores mais flexíveis e fáceis de usar
- **Factory methods** permitem criar objetos de diferentes maneiras

Melhores Práticas

Mantenha o construtor simples

Foque apenas na inicialização de atributos, evitando lógica complexa

Cuidado com objetos mutáveis

Nunca use listas ou dicionários vazios como valores padrão de parâmetros

Use nomes descritivos

Escolha nomes claros para parâmetros e atributos para melhorar a legibilidade